

IPA Project Report (Phase II)

Implementation of a 64-bit RISC-V CPU

Ritvik Avula
Roll No: 2024112019

Pavani Malchuru
Roll No: 2024102043

Nikhilesh Nallavelli
Roll No: 2024102051

Abstract—This report presents the next phase in the design and implementation of a 64-bit RISC-V CPU using Verilog. In this report, we provide a comprehensive overview of the pipelined architecture and detail the different kinds of hazard handling mechanisms that were implemented in the project.

I. INTRODUCTION AND BACKGROUND

THE RISC-V ISA is an open source instruction set architecture that focuses on maintaining simple hardware and using basic instructions in order to design and program more complicated entities. This simplicity makes RISC-V an ideal choice for custom processor design. The instruction set is modular, allowing implementations to support only the necessary base instructions (RV64I) or extend with optional extensions for floating-point operations, atomic instructions, and more. In this project, we focus on implementing the RV64I base integer instruction set for 64-bit systems.

Of all the instructions in the base RV64I ISA, we've implemented the following subset of instructions:

- **R-type** (add, add/or/xor, sll/srl/sra)
- **I-type** (addi and etc..., ld, jalr)
- **S-type** (sd)
- **B-type** (beq)
- **J-type** (jal)

II. OVERVIEW OF THE PIPELINED ARCHITECTURE

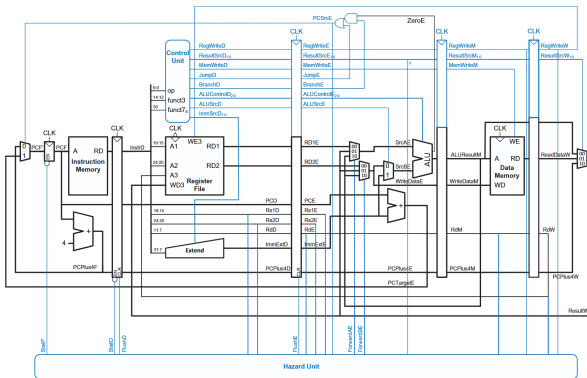


Fig. 1: High-level architecture of the RV64I pipelined processor. Courtesy of *Digital Design and Computer Architecture*, [1]

The RV64I CPU consists of several key modules that work in conjunction to execute instructions. Fig. 1 illustrates the high-level architecture of the processor.

The datapath consists of the following main components:

- 1) **Program Counter (PC)**: Stores the address of the current instruction and increments to fetch the next instruction
- 2) **Instruction Memory**: Stores the program instructions
- 3) **Register File**: Contains 32 general-purpose 64-bit registers
- 4) **Arithmetic Logic Unit (ALU)**: Performs computations - either arithmetic or to declare flags to be used in conditional operations
- 5) **Control Unit**: Decodes and instruction and generates control signals that determine which modules should be active during the instruction cycle
- 6) **Immediate Generator**: Extracts and extends immediate values from instructions

In this part of the project, we've added a **Hazard Detection Unit** to account for various hazards that pop up when pipelining a processor - namely hazards like:

- 1) **Data Hazards**: Resolves most RAW data hazards by introducing forwarding paths from MEM and WB stages to the EX stage inputs.
- 2) **Control Hazards**: Detects branch control hazards and flushes the buffers between stages if needed.

III. SOLVING HAZARDS

This section goes into more detail about how different types of hazards are solved in the pipelined CPU.

A. Data Hazards

Data hazards occur when an instruction depends on the result of a previous instruction that has not yet completed executing. In a pipelined processor, multiple instructions are in different stages simultaneously - which can lead to situations where data is read before it has been written.

In our version of a pipelined CPU, we majorly try to solve **Read-After-Write (RAW) Hazards** - In this type of data hazard, an instruction tries to read a register before a previous instruction writes to it.

1) **RAW Hazards and Forwarding**: The textbook *Computer Organization And Design* [2] describes two major pairs of RAW hazards:

- 1a. EX/MEM.RegisterRd = ID/EX.RegisterRs1
- 1b. EX/MEM.RegisterRd = ID/EX.RegisterRs2
- 2a. MEM/WB.RegisterRd = ID/EX.RegisterRs1
- 2b. MEM/WB.RegisterRd = ID/EX.RegisterRs2

Consider the following instruction sequence:

```
add x5, x1, x2    # x5 = x1 + x2
sub x6, x5, x3    # x6 = x5 - x3
```

Without hazard handling, when `sub` reaches the EX stage and attempts to read `x5`, the `add` instruction may still be in the MEM or WB stage, meaning the updated value of `x5` has not yet been written back to the register file.

We can fix these problems by **forwarding the data**. The forwarding unit compares the source registers of the instruction in the EX stage (`rs1_E`, `rs2_E`) against destination registers in later stages (`rd_M` in MEM stage and `rd_W` in WB stage). When a match is detected and the corresponding `RegWrite` signal is enabled, the forwarding unit selects the computed value directly from the later stage instead of using the stale value from the register file.

Two forwarding select signals are generated:

- **ForwardA:** Controls which value feeds ALU input A.
 - 00: Use register file output (no hazard)
 - 01: Forward from WB
 - 10: Forward from MEM
- **ForwardB:** Controls which value feeds ALU input B. (same encoding)

Note that when comparing register addresses, both forwarding paths activity, we give priority to the value in MEM rather than WB because it contains the more recent result.

Hence, the code snippet looks like the following:

```
if(((rs1_e == rd_m) && reg_write_m) &&
    (rs1_e != 0)) // EX-MEM hazard
    forward_ae = 2'b10;
else if(((rs1_e == rd_w) && reg_write_w)
    && (rs1_e != 0)) // MEM-WB hazard
    forward_ae = 2'b01;
else
    forward_ae = 2'b00;
```

Of course, the same follows for `rs2` and `forward_be`.

A Note on Load-to-Store Forwarding

An important special case occurs when a load instruction is immediately followed by a store instruction that uses the loaded value:

```
ld x5, 0(x1)    # Load x5 from memory
sd x5, 8(x2)    # Store x5 to different
                # address in memory
```

In this scenario, the store instruction in the Memory stage needs the value from `x5` as its write data. Since the load result becomes available at the end of the Memory stage (or from the MEM/WB pipeline register), we implement a forwarding path that routes the loaded data directly to the store's write data input. This eliminates the need for the data to first pass through the Writeback stage and register file, allowing back-to-back load-store operations without stalling.

This forwarding mechanism eliminates most RAW hazards without stalling the pipeline.

2) **Load-Use Hazards and Stalling:** Forwarding cannot solve all RAW hazards. A critical exception is the **load-use hazard**, where a load instruction is immediately followed by an instruction that uses the loaded data:

```
ld x5, 0(x1)    # Load x5 from memory
add x6, x5, x2   # Immediately use x5
```

The problem is that the data from memory becomes available only at the end of the MEM stage, but the `add` instruction needs it at the beginning of the EX stage—one cycle too early. Forwarding paths exist, but the data simply isn't ready yet.

To fix this, we used a **Hazard Detection Unit** that identifies load-use hazards by checking:

- If the instruction in the EX stage is a load (`MemRead_E = 1`)
- If its destination register (`rd_E`) matches either source register of the instruction in the ID stage (`rs1_D` or `rs2_D`)

When a load-use hazard is detected, the control unit:

- 1) Stalls the IF and ID stages for one cycle
- 2) Inserts a NOP (bubble) into the EX stage by clearing all control signals
- 3) After the one-cycle stall, normal forwarding can supply the loaded data from the MEM stage

The code looks as follows.

```
assign lw_stall = mem_to_reg_e[0] &
                (rd_e != 0) &&
                ((rs1_d == rd_e) || (rs2_d
                == rd_e));
assign stall_d = lw_stall;
assign stall_f = lw_stall;
```

This single-cycle penalty is unavoidable for load-use hazards but minimizes performance impact compared to more conservative stalling strategies.

B. Control Hazards

Control hazards (also called branch hazards) occur when the pipeline must make decisions about instruction flow before determining whether a branch will be taken. In a pipelined processor, by the time a branch instruction reaches the stage where its condition is evaluated, several subsequent instructions have already been fetched into the pipeline. If the branch is taken, these speculatively fetched instructions are incorrect and must be discarded.

Consider this example:

```
beq x1, x2, target    # Branch if x1 == x2
add x3, x4, x5         # Next sequential
                      # instruction
sub x6, x7, x8         # Another sequential
                      # instruction
target:
or x9, x10, x11        # Branch target
```

When `beq` enters the pipeline, the processor immediately fetches `add` and `sub` in subsequent cycles. However, if the branch is taken (`x1` equals `x2`), those instructions are incorrect and the pipeline must be flushed and restarted from `target`.

The most common type of branch prediction that requires minimal hardware is to just assume that the **branch will not be taken**. This is pretty good on its own - however, it fails miserably when loops come into picture. Hence, we're using a 2-bit dynamic branch predictor - this helps us predict branches with almost 90% accuracy.

1) *2-Bit Dynamic Branch Prediction*: To minimize the performance impact of control hazards, we implement a **2-bit dynamic branch predictor**. This predictor maintains a small table (Branch History Table or BHT) indexed by the lower bits of the branch instruction's PC. Each entry stores a 2-bit saturating counter representing one of four prediction states:

- **Strongly Not Taken (00)**: Predict branch will not be taken
- **Weakly Not Taken (01)**: Predict branch will not be taken, but less confident
- **Weakly Taken (10)**: Predict branch will be taken, but less confident
- **Strongly Taken (11)**: Predict branch will be taken

The state machine transitions based on the actual branch outcome:

- When a branch is **taken**, the counter increments (saturates at 11)
- When a branch is **not taken**, the counter decrements (saturates at 00)

The key advantage of the 2-bit predictor over a simpler 1-bit predictor is **tolerance to one-time branch behavior changes**. For example, a loop branch that is taken 99 times and not taken once (to exit the loop) will only mispredict twice with a 2-bit predictor: once when entering the loop and once when exiting. A 1-bit predictor would mispredict on every loop iteration.

2) *Branch Misprediction Recovery*: When the branch condition is evaluated in the Execute stage and the actual outcome differs from the prediction:

- 1) The predictor's state is updated based on the actual outcome
- 2) All instructions fetched after the branch (in IF and ID stages) are **flushed** by converting them to NOPs
- 3) The PC is corrected to either the branch target (if taken) or the sequential instruction (if not taken)
- 4) The pipeline continues from the correct address

The cost of a misprediction is typically 2–3 cycles (depending on when the branch is resolved), compared to always stalling for every branch, which would incur the penalty on every branch instruction regardless of the outcome.

Our implementation maintains high prediction accuracy for programs with regular branch patterns (loops, frequently taken or not-taken branches), significantly reducing the average branch penalty compared to static prediction schemes.

IV. IMPLEMENTATION DETAILS

A. Structure

Each module of the CPU has its own designated folder, containing:

- The file containing the module itself (<module>.v)

- A testbench to validate each of the functions of that module (<module>_tb.v)
- A bunch of intermediary files (.vvp, .vcd) that plot outputs

Apart from these folders, we also have a folder for testcases, written in assembly(.asm files) - however, .txt files also work.

1) *Assembler and its Features*: We've also written a custom assembler in C - to translate RISC-V assembly code into machine code. It reads assembly instructions from .asm or .txt files, parses each instruction, and generates the corresponding 32-bit machine code that can be executed by the CPU.

The Assembler supports the following key features:

- **Instruction Parsing**: Recognizes all R-type, I-type, S-type, B-type, and J-type instructions implemented in the CPU
- **Register Mapping**: Only the general naming convention (x0, x5, etc.) works.
- **Immediate Encoding**: Encodes immediate values and performs sign extension where necessary
- **Machine Code Generation**: Outputs 32-bit instruction encodings in a format compatible with the instruction memory module

The assembler operates in two passes:

- 1) **First Pass**: Scans the assembly file to identify and record the addresses of all labels
- 2) **Second Pass**: Translates each instruction to machine code, using the label table to resolve branch and jump offsets

The output is written to a text file where each line represents one byte of the instruction in hexadecimal format.

And finally, the folder also contains a script that assembles the .asm files, runs them on the CPU and generates all the required output files.

This project successfully demonstrates the design and implementation of a 64-bit RISC-V processor core. The modular architecture allows for clear separation of concerns and facilitates future extensions. All modules have been thoroughly tested and verified to operate correctly. The Phase 1 design provides a solid foundation for implementing a pipelined processor in subsequent phases.

REFERENCES

- [1] S. Harris and D. Harris, *Digital Design and Computer Architecture, RISC-V Edition*. Morgan Kaufmann, 2021.
- [2] D. A. Patterson and J. L. Hennessy, *Computer organization and Design*. Morgan Kaufmann, 2022.
- [3] J. L. Hennessy and D. A. Patterson, *Computer architecture: a quantitative approach*. Elsevier, 2011.
- [4] DownToTheWires, "Risc-v intro and r-type alu instructions," 2025, accessed: 2025. [Online]. Available: <https://www.youtube.com/@Downtothewires>